FACULTY OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY (FSKTM)

BIC21203 WEB DEVELOPMENT

SEMESTER 1/20262027

ASSIGNMENT TITLE :

Lab 4B

INSTRUCTOR :

DR. ROSMAMALMI BINTI MAT NAWI

NAME :	NUR SHAHIRAH BINTI MOHAMAD NAZIM
MATRIC NO. :	CI240091
SECTION :	SECTION 4

Lab #4b:
CLIENT-SIDE SCRIPTING (JSON and jQuery)

Topic	**Web page development using JSON, modern JavaScript, and jQuery**
Domain of Learning	Psychomotor (P2: Set; P3: Guided Response; P4: Mechanism)
Learning Objective	Retrieve, parse, access, and display JSON data using modern JavaScript; compare native DOM operations with jQuery.
Lab Activity Objective	Create a valid JSON dataset, load and display selected data using fetch(), and implement equivalent DOM updates and events using jQuery.

Instructions

a. Complete Questions 1–3 individually.
b. Use a code editor, a modern browser, and a local development server (e.g., VS Code Live Server) to test fetch() with local JSON files. Do not open the HTML directly using file:// for Question 2.
c. Use separate files named books.json, books.html, and jquery-dom.html. Push your work to a GitHub repository, including a README.md with setup instructions and screenshots.
d. Submit a concise PDF report through AUTHOR containing the required outputs, brief explanations, and your GitHub repository URL.

Question 1 — Create and Structure JSON Data (5 marks)

Consider the following collection of books:

Property	**Book 1**	**Book 2**
title	The Great Gatsby	To Kill a Mockingbird
author	F. Scott Fitzgerald	Harper Lee
publicationYear	1925	1960
genres	Fiction, Classics	Fiction, Coming-of-Age

Create a valid file named books.json containing a top-level "books" array with two book objects. Each book must include title (string), author (string), publicationYear (number), and genres (array of strings). Ensure the JSON uses double-quoted property names and has no trailing commas.

Expected evidence:
A screenshot of the complete JSON file and confirmation that it parses successfully.

Answer Question 1



Figure 1.1: books.json File

Figure 1.1 shows the JSON file named books.json contains a top-level books array with two book objects which is The Great Gatsby and To Kill a Mockingbird. Each book object consists of four properties, title, author, publicationYear and genres. The title and author are stored as strings, publicationYear is stored as a number, and genres is stored as an array of strings. The JSON structure uses double quotation marks and follows the correct syntax without trailing commas.

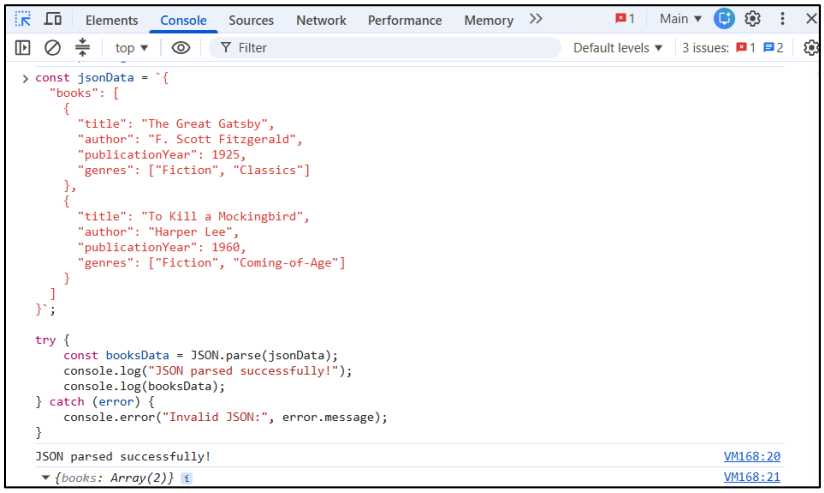


Figure 1.2: JSON Successfully Parse using JavaScript JSON.parse()

Figure 1.2 shows the validation of the JSON data using JavaScript in the browser's Developer Tools Console. The JSON.parse() method is used to convert the JSON string into a JavaScript object. A try-catch

statement is also implemented to handle any parsing errors. The console displays the message "JSON parsed successfully!" and shows the books array containing two book objects. This confirms that the JSON data has valid syntax and can be successfully parsed by JavaScript.

Question 2 — Retrieve and Display JSON Data (5 marks)

Create books.html. Using the native JavaScript fetch() API, read books.json and display the following items within the webpage (not only in the browser console):

(a) Title of the first book. (b) Author of the second book. (c) Genres of both books, clearly grouped by book title.

Use textContent and/or createElement() to display text safely. Handle unsuccessful requests or invalid JSON by displaying a user-friendly error message. You may use .then() or async/await. Run the HTML through a local server and capture the browser output.

Expected output:
The Great Gatsby; Harper Lee; The Great Gatsby — Fiction, Classics; To Kill a Mockingbird — Fiction, Coming-of-Age.

Answers Question 2

books.html > html > body > script > catch() callback

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Book Information</title>
</head>
<body>
    <h1>Book Information</h1>
    <div id="bookDetails">
        <p>Loading book information...</p>
    </div>
    <script>
        // retrieve json data using fetch()
        fetch("books.json")
            .then(response => {
                // check request successful
                if (!response.ok) {
                    throw new Error("Unable to retrieve book data.");
                }

                // convert json response into JavaScript object
                return response.json();
            })
            .then(data => {
                const container = document.getElementById("bookDetails");

                // display title first book
                const firstTitle = document.createElement("p");
                firstTitle.textContent = "First Book Title: " + data.books[0].title;
                container.appendChild(firstTitle);

                // display author second book
                const secondAuthor = document.createElement("p");
                secondAuthor.textContent = "Second Book Author: " + data.books[1].author;
                container.appendChild(secondAuthor);

```

```
            // display genres of both books
            const genreHeading = document.createElement("h2");
            genreHeading.textContent = "Genres of Both Books";
            container.appendChild(genreHeading);

            data.books.forEach(book => {
                const bookGenre = document.createElement("p");
                bookGenre.textContent =
                    book.title + " — " + book.genres.join(", ");
                container.appendChild(bookGenre);
            });
        })
        .catch(error => {
            //dDisplay userfriendly error message
            const container = document.getElementById("bookDetails");
            container.textContent = "Error: Unable to load book information. Please try again.";
            console.error(error);
        });
    </script>
</body>
</html>
```

Figure 2.1: books.html File

Figure 2.1 shows the HTML and JavaScript file to retrieve data from the book.json file. The native JavaScript fetch() API was used to request the JSON data, while the response.json() method converted the response into a JavaScript object. The required information was accessed using array indexing and object properties.

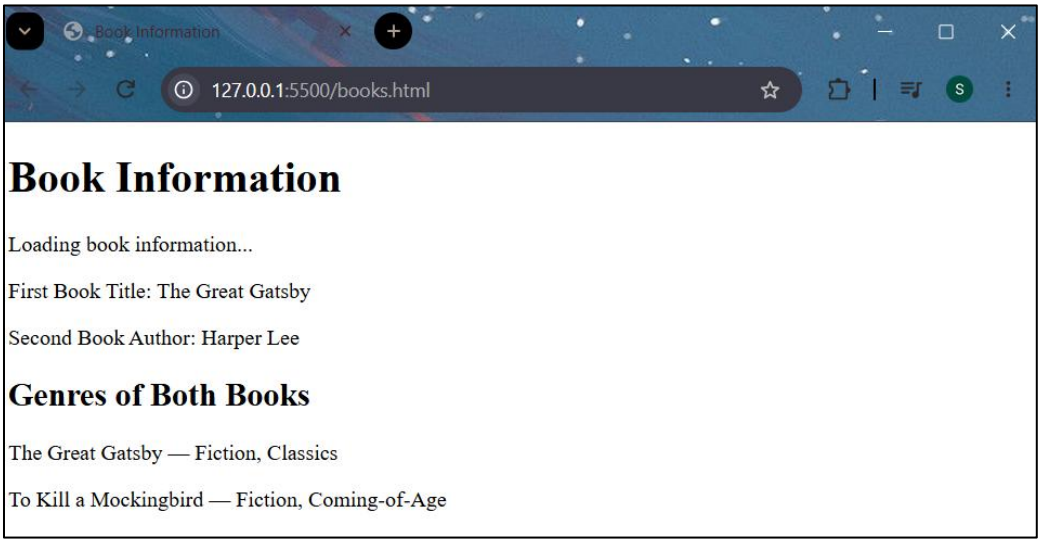


Figure 2.2: Browser Output Automatically Open Using Visual Studio Code Live Server

Figure 2.2 shows the retrieved information was displayed on the webpage using createElement(), textContent, and appendChild(). The webpage shows the title of the first book, the author of the second book, and the genres of both books grouped by their respective titles. A .catch() method was also implemented to handle errors and display a user-friendly message if the JSON data could not be retrieved or parsed.

Question 3 — Compare Native JavaScript and jQuery (5 marks)

Create jquery-dom.html. Include jQuery 4.0.0 using the official full-build CDN link below (not the slim build):

https://code.jquery.com/jquery-4.0.0.min.js

Use the following HTML elements and CSS class:

```
<div id="myElement">Original Text</div>

<div id="myDIV"><p>I am a myDIV.</p></div>

<button id="myButton">Click me</button>

.myStyle { background-color: coral; padding: 16px; }
```

Write the jQuery equivalents for all three native JavaScript operations below. The result must (a) replace the text of #myElement with "Modified Text", (b) add the myStyle class to #myDIV, and (c) respond to a click on #myButton by showing "Button clicked!".

```
// (a) document.getElementById("myElement").textContent = "Modified Text";

// (b) document.querySelector("#myDIV").classList.add("myStyle");

// (c) document.getElementById("myButton").addEventListener("click", () =>
alert("Button clicked!"));
```

Use .text(), .addClass(), and .on("click", ...). In your PDF report, briefly state one advantage of native JavaScript and one situation where an existing project may still use jQuery.

Answers Question 3

jquery-dom.html > html > body > div#myDIV > p

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>jQuery DOM Manipulation</title>

    <style>
        .myStyle {
            background-color: coral;
            padding: 16px;
        }
    </style>

    <!-- jQuery 4.0.0 Full-Build CDN -->
    <script src="https://code.jquery.com/jquery-4.0.0.min.js"></script>
</head>
<body>
    <h1>jQuery DOM Manipulation</h1>
    <div id="myElement">Original Text</div>
    <br>
    <div id="myDIV">
        <p>I am a myDIV.</p>
    </div>
    <br>
    <button id="myButton">Click me</button>
```

```
    <button id="myButton">Click me</button>
    <script>
        // wait until the document is ready
        $(document).ready(function() {

            // (a) Change the text of myElement
            $("#myElement").text("Modified Text");

            // (b) Add myStyle class to myDIV
            $("#myDIV").addClass("myStyle");

            // (c) Display alert when button is clicked
            $("#myButton").on("click", function() {
                alert("Button clicked!");
            });
        });
    </script>
</body>
</html>
```

Figure 3.1: jquery-dom.html File

Figure 3.1 shows implementation of the jquery-dom.html. The jQuery 4.0.0 library was included through the official CDN link. Three jQuery methods were implemented: .text() to modify the text of myElement, .addClass() to apply the myStyle CSS class to myDIV, and .on("click", ...) to handle the button click event. The document ready function ensures that the operations are executed after the webpage has loaded.

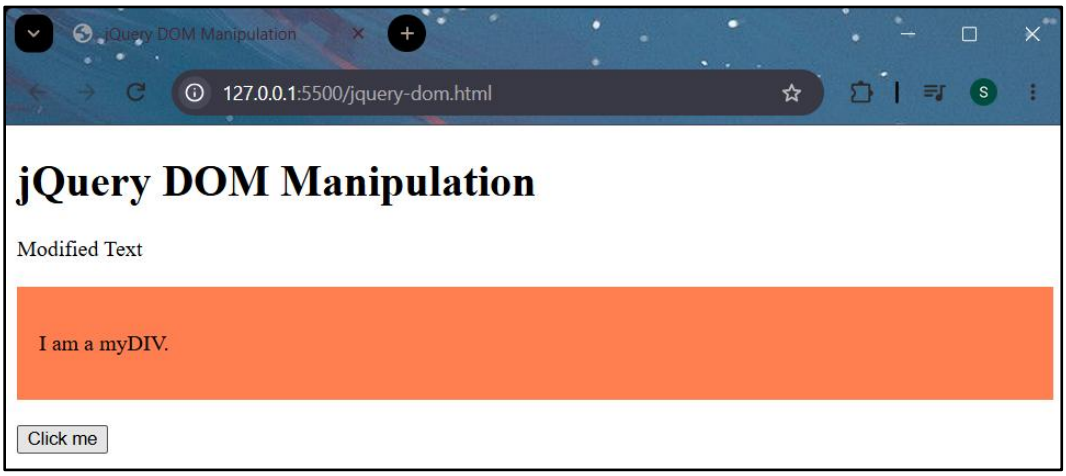


Figure 3.2: Browser Output Automatically Open Using Visual Studio Code Live Server

Figure 3.2 shows the webpage running successfully through Visual Studio Code Live Server. The original text was successfully changed to "Modified Text" using the .text() method. The myDIV element also displays a coral background with 16px padding after the .addClass() method was applied. This confirms that both jQuery DOM manipulation operations work correctly.

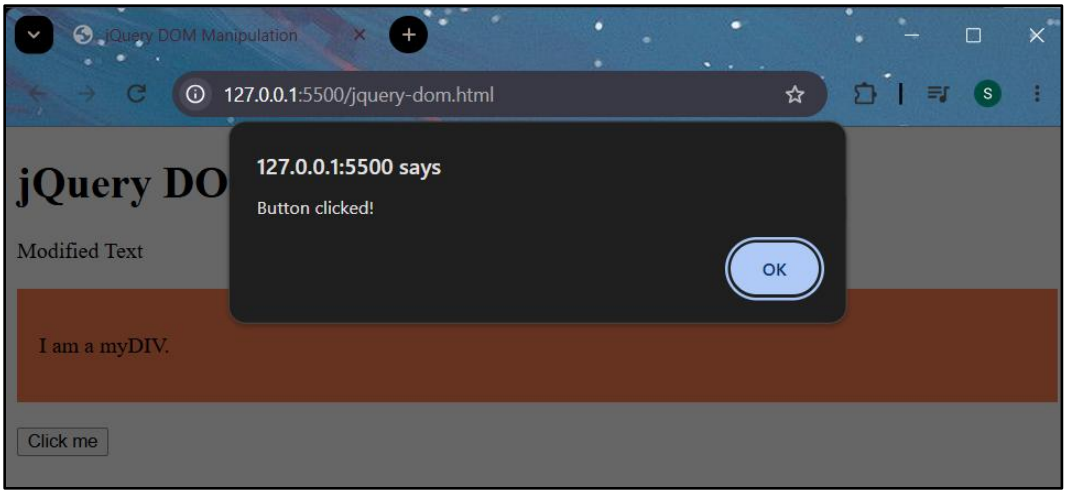


Figure 3.3: Alert Display "Button Clicked!" when "Click me" Button

Figure 3.3 shows the result of the button click event using the jQuery .on("click", ...) method. When the user clicks the "Click me" button, a JavaScript alert box appears with the message "Button clicked!". This confirms that the jQuery event handling operation works correctly.

Comparison Between Native JavaScript and jQuery

Native JavaScript	jQuery
Built directly into browsers	Requires the jQuery library
Does not require an external library	Provides shorter and simpler syntax for common DOM operations
Uses methods such as textContent and addEventListener()	Uses methods such as .text() and .on()

An existing project may still use jQuery when maintaining an older website that already depends on the jQuery library. Using jQuery allows developers to maintain the existing code without rewriting all DOM manipulation and event-handling operations using native JavaScript.

GitHub Link : https://github.com/shahirahnazim/BIC21203-WEB-DEVELOPMENT---LAB-4-B.git

Submission Checklist

- books.json and books.html work through a local server
- jquery-dom.html works correctly
- GitHub repository and README.md included □ PDF contains screenshots, explanations, and repository link

Assessment:

Assessment A — JSON Data Handling and jQuery Integration (CLO2 — Psychomotor)

Scoring: For each criterion, select a performance level (0–5) and calculate weighted marks as (score ÷ 5) × 5. Total: 15 marks.

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (1–2)	Unsatisfactory (0)	Marks
Parsing JSON Data (P4)	Valid, correctly typed JSON with an accurate books array and genre arrays.	Valid JSON with minor structure or typing issues.	Mostly valid structure; some data types or fields incorrect.	Incomplete JSON or significant structural errors requiring guidance.	No usable JSON dataset.	___ /5 × 5
Manipulating JSON Data (P4)	Successfully fetches JSON and correctly displays all required data with error handling.	Retrieves and displays all items with minor presentation or error-handling issues.	Retrieves JSON and displays some required data correctly.	Partial retrieval or incorrect item selection; substantial assistance needed.	No functional JSON retrieval or display.	___ /5 × 5
jQuery Integration for DOM and Events (P4)	All three operations work correctly using appropriate jQuery methods.	All three mostly work with minor implementation issues.	Two of three operations work correctly.	Only one operation works or methods are used incorrectly.	No functional jQuery operations.	___ /5 × 5

CLO2 Total: _____ /15

Assessment B — Presentation and Understanding (CLO3 — Affective)

Scoring: Clarity and Confidence = (score ÷ 5) × 2; Explanation = (score ÷ 5) × 3. Total: 5 marks.

Criteria	**Excellent (5)**	**Good (4)**	**Satisfactory (3)**	**Needs Improvement (1–2)**	**Unsatisfactory (0)**	**Marks**
Clarity and Confidence (A1)	Demonstrates outputs clearly, confidently, and independently.	Clear demonstration with minor hesitation.	Adequate demonstration with some uncertainty.	Unclear demonstration needing repeated prompting.	Unable to demonstrate the work.	___ /5 × 2
Explanation of JSON, JavaScript and jQuery (A2)	Explains JSON structure, fetch(), and jQuery operations accurately, with reasons for choices.	Explains most concepts correctly with minor gaps.	Provides a basic explanation of key concepts.	Limited explanation and cannot justify several choices.	Unable to explain relevant concepts.	___ /5 × 3

CLO3 Total: _____ /5 **OVERALL: _____ /20**

Lecturer note: The original rubric criteria and weightings are retained. Descriptors are updated to match the revised Questions 1–3. Verify P/A levels against the approved course CLO mapping before use.